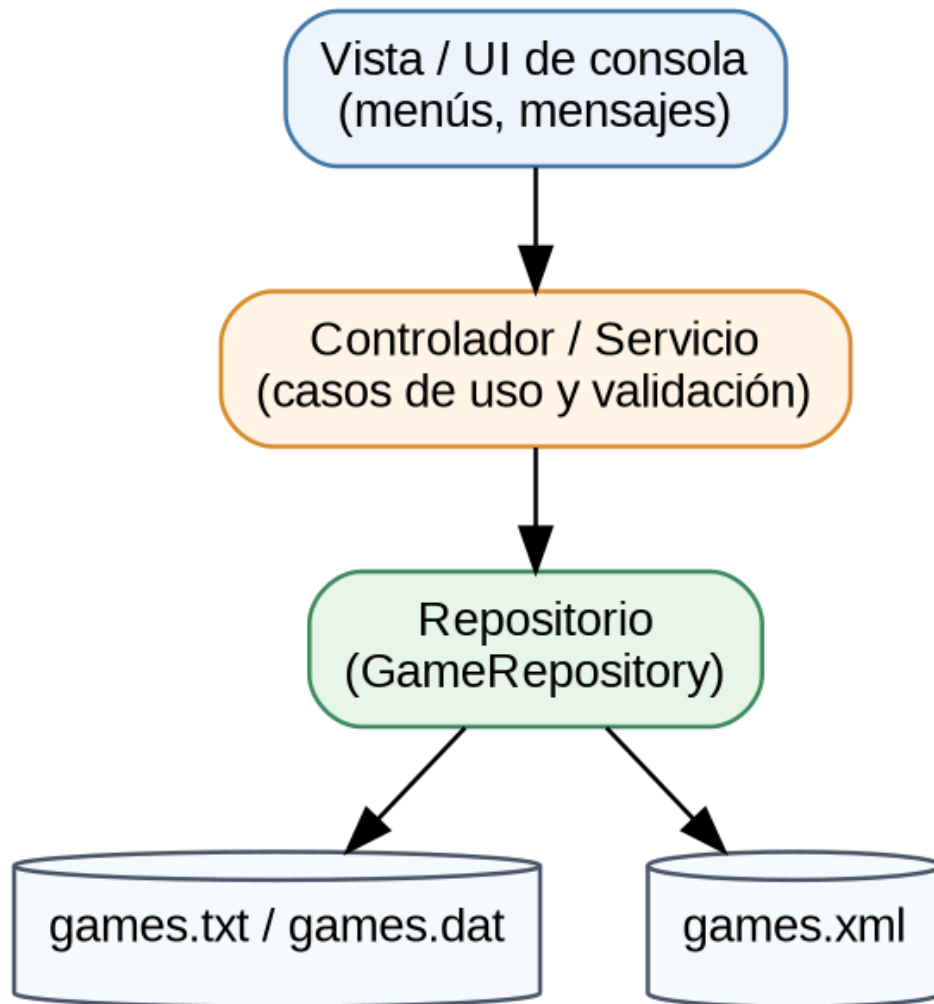


UNIDAD DE TRABAJO 1

Manejo de ficheros en Java

Acceso a Datos · 2º DAM



Aplicación transversal: GameHub. La persistencia irá evolucionando durante el módulo.

0. Qué vas a aprender en esta unidad	4
1. Ficheros, registros y persistencia	4
1.1. Clasificaciones útiles	5
1.2. Operaciones habituales con fichero	5
2. Gestión de archivos y directorios	6
2.1. La clase File	6
3. Acceso secuencial y acceso aleatorio	10
4. Flujos (streams) de entrada y salida. Secuencial.	11
5. Ficheros con Flujo de Caracteres. Ficheros de Texto	12
5.1. FileReader / FileWriter. Lectura carácter a carácter.	13
5.2. BufferedReader / BufferedWriter. Lectura y escritura por líneas	16
5.3. Sobrescribir o añadir	17
5.4. Clase Scanner	17
5.5. try-with-resources	17
6. Ficheros binarios y tipos primitivos	20
6.1. FileInputStream / FileOutputStream	20
6.2. DataInputStream y DataOutputStream	22
7. Serialización de objetos. ObjectOutputStream / ObjectInputStream.	26
8. Ficheros de acceso aleatorio: RandomAccessFile	28
8.1. Diseñar un registro de tamaño fijo	29
8.2. Escribir y leer registros	30
9. MVC. Modelo - Vista - Controlador	31

0. Qué vas a aprender en esta unidad

Esta unidad introduce la persistencia basada en ficheros: gestionar archivos y directorios, elegir entre acceso secuencial y aleatorio, trabajar con flujos de caracteres y bytes, serializar objetos y procesar formatos de intercambio como XML. El objetivo no es memorizar clases aisladas, sino aprender a diseñar una capa de acceso a datos clara, comprobable y sustituible.

Referencia curricular El RA1 del módulo pide desarrollar aplicaciones que gestionen información almacenada en ficheros, usar clases de gestión de ficheros y directorios, valorar formas de acceso, almacenar y recuperar información, convertir formatos, gestionar excepciones y probar/documentar las aplicaciones.

1. Ficheros, registros y persistencia

Un **fichero** o archivo es la agrupación de una serie de datos, almacenados en un soporte magnético para poder manipularla en cualquier momento. A diferencia de las variables en memoria, sus datos permanecen después de finalizar el programa.

Cada dato almacenado en un fichero se denomina **registro**. Un registro está formado, a su vez, por uno o varios datos que pueden ser de diferentes tipos. Cada dato que contiene el registro se le conoce con el nombre de **campo**.

Ejemplo En GameHub, un videojuego puede ser un registro. Sus campos podrían ser id, título, plataforma, género y horasJugadas.

Aunque parece ser que el uso de ficheros planos está obsoleto frente al uso de BBDD, el uso de ficheros se sigue utilizando en todo tipo de sistemas:

- Todavía hay sistemas que utilizan ficheros planos como almacenamiento de información en vez de una BBDD.
- **Mucha** de la información que se **envía por la red no deja de ser un texto plano (xml, txt)** y es necesario tener herramientas para poder acceder/modificar su contenido.
- Muchas veces nos facilita el desarrollo poder modificar y automatizar algunos procesos que están relacionados con ficheros: almacenar **temporalmente** datos, generar la entrada para otro sistema, descargar información de una API y filtrarla en un fichero...

1.1. Clasificaciones útiles

Criterio	Tipos	Idea clave
Contenido	Texto / binario	Texto: representa caracteres. Binario: bytes con una estructura definida por el formato o por el programa.
Forma de acceso	Secuencial / aleatorio	Secuencial: se avanza desde el principio. Aleatorio: se salta a una posición concreta.
Estructura	Plano / estructurado	Plano: líneas o campos simples. Estructurado: XML, JSON u otros formatos con estructura explícita.

Importante. Un fichero binario no “ocupa siempre menos” que uno de texto. El tamaño depende del formato, la codificación, la compresión y los datos. Tampoco es correcto afirmar que cada carácter de un .txt ocupa siempre 1 byte: en UTF-8 puede ocupar entre 1 y 4 bytes.

1.2. Operaciones habituales con fichero

- **Creación del fichero.** El fichero se crea en el disco con un nombre que después se debe usar para acceder a él. Este proceso solo se hace una vez.
- **Apertura del fichero.** Para que un programa pueda operar con un fichero, primero debe realizar su apertura. Para ello, el programa utilizará algún método para identificar el fichero, como usar un descriptor del fichero.
- **Cierre del fichero.** El fichero se debe cerrar cuando el programa no lo vaya a utilizar. Suele ser la última instrucción del programa.
- **Lectura de datos del fichero.** Este proceso consiste en transferir información del fichero a la memoria, a través de variables del programa.
- **Escritura de datos en el fichero.** Este proceso consiste en transferir información de la memoria, por medio de las variables del programa, al fichero.

Una vez abierto el fichero las operaciones típicas que se realizan sobre un fichero son las de un CRUD:

- **Altas.** Añadir un nuevo registro al fichero.
- **Bajas.** Eliminar un registro del fichero.
- **Modificaciones.** Cambiar parte del contenido de un registro. Antes de modificar, será necesario localizar el registro a modificar dentro del fichero.
- **Consultas.** Buscar en el fichero un determinado registro.

2. Gestión de archivos y directorios

2.1. La clase File

En Java la **clase File** (`java.io.File`) representa una ruta a un fichero o directorio y permite consultar propiedades y realizar algunas operaciones. Más información: <https://docs.oracle.com/javase/8/docs/api/java/io/File.html>

Constructor	Descripción
<code>File(String directorioYFichero)</code>	Recibe en la instanciación del objeto el camino completo donde está el fichero junto con el nombre. Por defecto, si no se indica, lo busca en la carpeta del proyecto. <code>directorioYFichero</code> puede ser también el camino a un directorio, sin indicar al final el nombre de ningún fichero.
<code>File(String directorio, String nombreFichero)</code>	Recibe en la instanciación del objeto el camino completo donde está el fichero, como primer parámetro, y el nombre del fichero como segundo parámetro.
<code>File(File directorio, String fichero)</code>	Recibe en la instanciación del objeto un objeto de tipo <code>File</code> , que hace referencia a un directorio, como primer parámetro y el nombre del fichero como segundo parámetro.

Hay que tener en cuenta que se puede:

- ▶ Indicar el nombre de un fichero sin la ruta, se buscará el fichero en el directorio actual
- ▶ Indicar el nombre de un fichero con la **ruta relativa** ("/ficheros/fichero.txt").
- ▶ Indicar el nombre de un fichero con la **ruta absoluta** ("C:/Windows/Usuario/Escritorio/[...]/ficheros/fichero.txt").

Constructor	Descripción
boolean canRead() boolean canWrite()	Informa si se puede leer la información que contiene o si se puede escribir en él.
boolean exists()	Informa si el fichero o el directorio existe
boolean isDirectory()	Devuelve true si el objeto File corresponde a un directorio.
boolean isFile() boolean isDirectory()	Devuelve true si el objeto File corresponde a un fichero normal, o si es un directorio
long lastModified()	Devuelve la fecha de la última modificación.
String getName()	Devuelve el nombre del fichero o directorio.
String getPath()	Devuelve la ruta relativa.
String getAbsolutePath()	Devuelve la ruta absoluta.
String getParent()	Devuelve el nombre del directorio padre o null si no existe

Constructor	Descripción
delete()	Borra el fichero.
long length()	Devuelve el tamaño del archivo en bytes.
boolean renameTo(File)	Cambia el nombre del fichero.
boolean mkdir()	Crea el directorio
String[] list()	Devuelve un listado de los archivos que se encuentran en el directorio

AC1.2. Realiza un programa al que se le pase la ruta de un determinado fichero o directorio y nos muestre la siguiente información. Nombre del fichero, si el fichero existe o no, la ruta absoluta, si se puede escribir, si se puede leer, el tamaño, si es un directorio y si es un fichero.

```

---- Información del fichero: BUILD.XML
El fichero SI existe
Ruta Absoluta      : /Users/carlos/NetBeansProjects/AC1.2/build.xml
Se puede leer      : true
Se puede escribir   : true
Tamaño del fichero : 3527
Es un directorio    : false
Es un fichero       : true
BUILD SUCCESSFUL (total time: 0 seconds)

```

AC1.3. Realiza un programa que muestre el contenido de un determinado directorio. Usar solo la clase java.io.File.

```

Contenido del Directorio: AC1.3
|---->Fichero: .DS_Store
|---->Directorio: build
|---->Fichero: build.xml
|---->Directorio: lib
|---->Fichero: manifest.mf
|---->Directorio: nbproject
|---->Directorio: src
|---->Directorio: test

```

Ejemplo: Listar el contenido de un directorio

```

import java.io.File;
import java.util.Scanner;

public class ListarArchivos {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce la ruta del directorio: ");
        String rutaDirectorio = scanner.nextLine();

        File fichero = new File(rutaDirectorio);

        if (fichero.isDirectory()) {
            for (File f : fichero.listFiles()) {
                System.out.println(f.getName() + " " + f.getAbsolutePath());
            }
        } else {
            System.out.println("No es un directorio!!");
        }

        scanner.close();
    }
}

```

```

import java.io.File;
import java.util.Scanner;

public class ListarArchivos {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Introduce la ruta del directorio: ");
            File directorio = new File(scanner.nextLine());

            if (!directorio.isDirectory()) {
                System.out.println("La ruta no es un directorio válido.");
                return;
            }

            File[] elementos = directorio.listFiles();
            if (elementos == null) {
                System.out.println("No se pudo leer el directorio.");
                return;
            }

            for (File elemento : elementos) {
                System.out.printf("%-30s %s\n",
                    elemento.getName(), elemento.getAbsolutePath());
            }
        }
    }
}

```

AC1.3. AMPLIACIÓN. Añadirle la opción de mostrar todo el árbol de directorios que contiene.

```

Contenido del Directorio: AC1.3
|---->Fichero: .DS_Store
|---->Directorio: build
|---->Fichero: build.xml
|---->Directorio: lib
|---->Fichero: manifest.mf
|---->Directorio: nbproject
|---->Directorio: src
|---->Directorio: test

```

```

TODO el contenido del Directorio: AC1.3
Directorio: AC1.3
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/.DS_Store
|---->Directorio: build
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/.DS_Store
|---->Directorio: classes
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/classes/.netbeans_automatic_build
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/classes/.netbeans_update_resources
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/classes/Directorio.class
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/classes/Main.class
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build/classes/MensajeTabulado.class
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/build.xml
|---->Directorio: lib
|---->Directorio: CopyLibs
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/lib/CopyLibs/org-netbeans-modules-java-j2seproject-copylibstask.jar
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/lib/nblibraries.properties
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/manifest.mf
|---->Directorio: nbproject
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/build-impl.xml
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/genfiles.properties
|---->Directorio: private
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/private/config.properties
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/private/private.properties
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/project.properties
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/nbproject/project.xml
|---->Directorio: src
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/src/Directorio.java
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/src/Main.java
|---->Fichero: /Users/carlos/NetBeansProjects/AC1.3/src/MensajeTabulado.java
|---->Directorio: test

```

AC1.4 – Realiza un programa que cree un directorio y un fichero en el mismo directorio del proyecto. A continuación pregunte si desea borrarlos, en caso afirmativo procede.

```

EL fichero ha sido creado con éxito
EL directorio ha sido creado con éxito
Desea borrar los elementos creados? (s/n): s
Borrando el Archivo .....
Archivo Borrado con Éxito
Borrando el Directorio .....
Directorio Borrado con Éxito

```

3. Acceso secuencial y acceso aleatorio



Figura 3. Diferencia conceptual entre acceso secuencial y aleatorio.

Aspecto	Secuencial	Aleatorio
Localizar registro	Se recorren registros hasta encontrarlo.	Se calcula/sabe la posición y se salta directamente.
Tamaño de registro	Puede ser variable.	Habitualmente fijo si calculamos posiciones por índice.
Altas	Sencillas al final.	Sencillas si hay hueco o posición definida.
Bajas/modificaciones	A menudo requieren reescritura en fichero auxiliar.	Pueden hacerse in-place si el registro tiene tamaño fijo.
Uso	Logs, exportaciones, lectura completa.	Índices simples, registros fijos, acceso por id.

4. Flujos (streams) de entrada y salida. Secuencial.

Para Java todo movimiento de datos lo considera un flujo, o un **stream**, y el tratamiento de un fichero no difiere de la obtención de información por el teclado o mostrar información por la salida estándar, es un **flujo externo de información**, o un conjunto de bytes almacenados en un dispositivo externo al programa.

- **Flujos de caracteres (16 bits)**: realizan operaciones de entradas y salidas de caracteres. El flujo de caracteres viene gobernado por las clases **Reader** y **Writer**.

- **Flujos de bytes (8 bits)**: realizan operaciones de entradas y salidas de bytes y su uso está orientado a la lectura y escritura de datos binarios. Todas las clases de flujos de bytes heredan las clases **InputStream** y **OutputStream**.

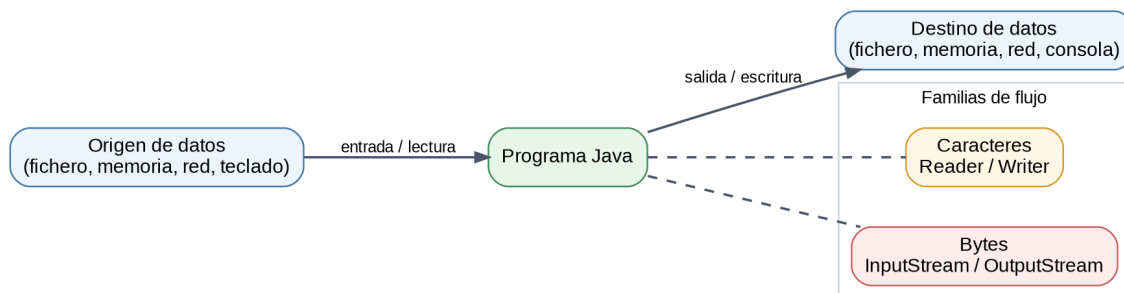
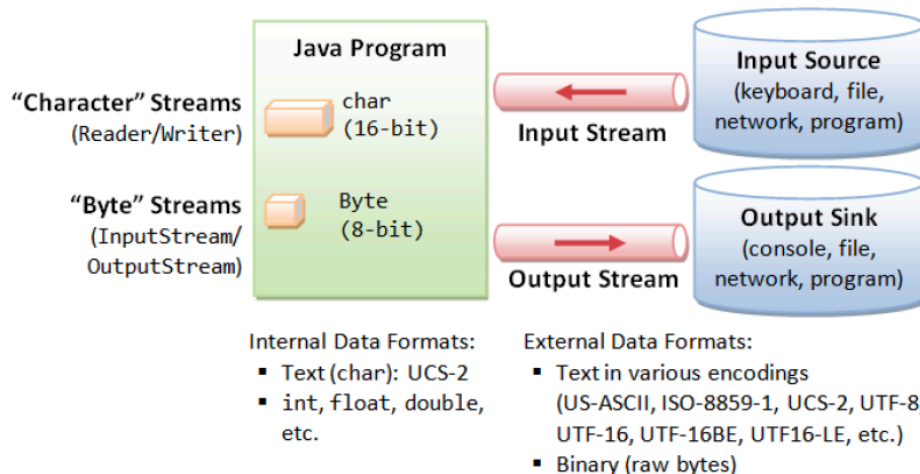
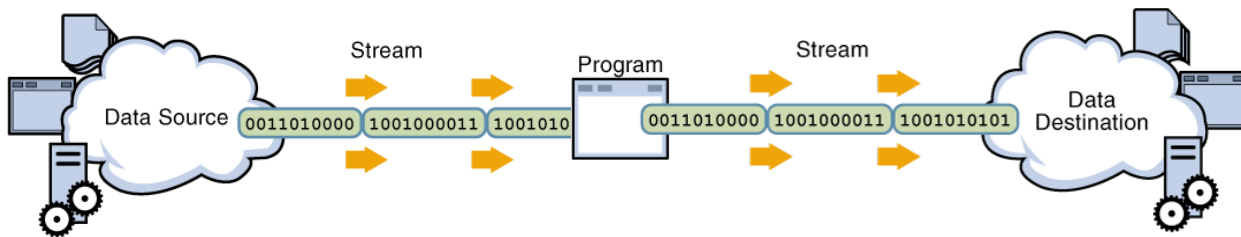


Figura 1. Modelo mental de entrada/salida y familias de streams.

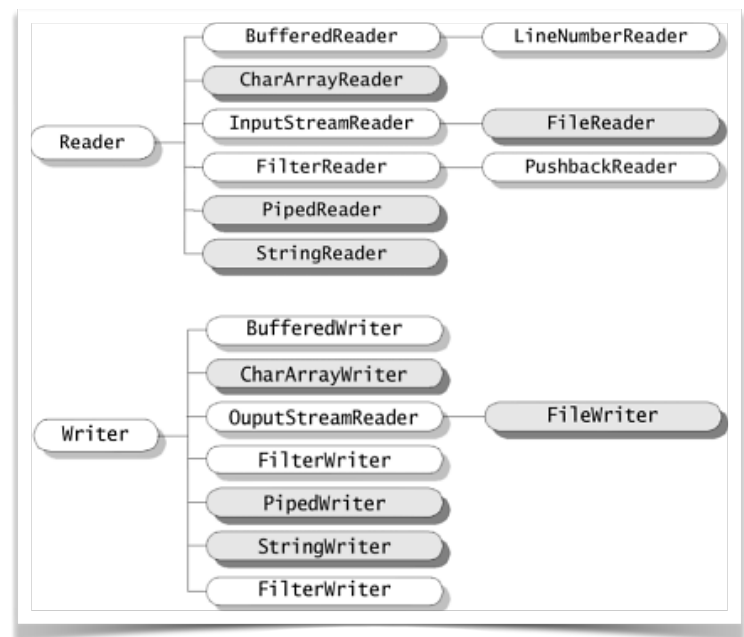
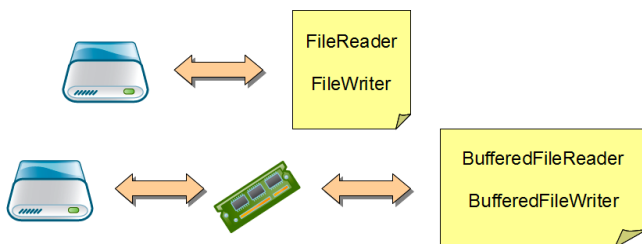
5. Ficheros con Flujo de Caracteres. Ficheros de Texto

Los ficheros de texto, que normalmente se generan con un editor de textos, almacenan caracteres alfanuméricos en un formato estándar (ASCII, Unicode, UTF-8, etc.). Para trabajar con ellos en Java, se utilizan la clase **FileReader** para leer caracteres y la clase **FileWriter** para escribir los caracteres en el fichero. Además, las lecturas o escrituras realizadas sobre un fichero deben escribirse dentro de un manejador de **excepciones try-catch**. Al instanciar un objeto con la clase **FileReader**, el programa abre el fichero que se envía como argumento para lectura y su información se puede leer de forma secuencial **carácter a carácter**.

Al usar **FileReader** / **FileWriter** puede lanzar varias **excepciones**:

- **FileNotFoundException**. Por que el nombre del fichero no exista o no sea válido.
- **IOException**. Indicando que el disco está lleno o protegido contra la escritura.

Los ficheros de texto son adecuados cuando queremos que los datos sean legibles y fáciles de intercambiar. Para ejercicios pequeños podemos usar **FileReader/FileWriter**; para trabajo por **líneas**, **BufferedReader/BufferedWriter** son más cómodos y eficientes.



5.1. FileReader / FileWriter. Lectura carácter a carácter.

Class FileReader

```
java.lang.Object
  java.io.Reader
    java.io.InputStreamReader
      java.io.FileReader
```

Class FileWriter

```
java.lang.Object
  java.io.Writer
    java.io.OutputStreamWriter
      java.io.FileWriter
```

Método	Explicación
FileReader(File fichero).	Lanza la excepción FileNotFoundException si el fichero pasado no existe
FileReader(String fichero)	Lanza la excepción FileNotFoundException si el fichero pasado no existe
int read()	Lee un carácter y lo devuelve.
int read(char[] buf)	Lee hasta buf.length caracteres de datos de la matriz buf pasada como parámetro
int read(char[] buf, int desplazamiento, int n)	Lee hasta n caracteres de datos de una matriz buf comenzando por buf[desplazamiento] y devuelve el número leído de caracteres.

Método	Explicación
FileWriter(String nombreFich) FileWriter(String nombreFich, boolean append)	Recibe como parámetro el nombre del fichero a abrir y borra el contenido previo del fichero comenzando a escribir desde el principio. Si append es true, se sitúa al final del fichero para añadir contenido desde el final.
FileWriter(File fichero) FileWriter(File fichero, boolean append)	Recibe como parámetro un objeto File que representa al fichero con el que queremos trabajar y borra el contenido previo del fichero comenzando a escribir desde el principio. Si append es true, se sitúa al final del fichero para añadir contenido desde el final.
void write(int c)	Escribe un carácter.
void write(char[] buf)	Escribe un array de caracteres.
void write(char[] buf, int desplazamiento, int n)	Escribe n caracteres de datos de una matriz buf comenzando por buf[desplazamiento].

Método	Explicación
void write(int c)	Escribe un carácter .
void write(char[] buf)	Escribe un array de caracteres.
void write(char[] buf, int desplazamiento, int n)	Escribe n caracteres de datos de una matriz buf comenzando por buf[desplazamiento].
void write(String str)	Escribe una cadena de caracteres.
append(char c)	Añade un carácter a un fichero.

Ejemplo: Lectura de texto carácter a carácter.

```
try (FileReader reader = new FileReader("entrada.txt", StandardCharsets.UTF_8)) {
    int dato;
    while ((dato = reader.read()) != -1) {
        char caracter = (char) dato;
        System.out.print(caracter);
    }
}
```

Por qué read() devuelve int Necesita representar todos los valores de char y además el valor especial -1 para indicar fin de flujo.

Ejemplo: Escritura / Lectura de un fichero de texto.

- Se crea un fichero (usando FileWriter) y un flujo para la escritura de una frase recogida por teclado.
- Se crea un flujo para la lectura de los datos, los cuales se leen carácter a carácter.

```
Scanner InputUser = new Scanner(System.in);
String frase, cadenaLeida = "";
int car; char cara; FileWriter fichEsc;
try {
    // Instanciamos el fichero para escribir en él
    fichEsc = new FileWriter("nuevo.txt");

    // Obtenemos una frase de teclado
    System.out.println("Introduce una frase: ");
    frase = InputUser.nextLine();

    // Escribimos la frase (string) en el fichero
    fichEsc.write(frase);
    fichEsc.close();

    // Instanciamos el fichero para leer de él
    FileReader fichLect = new FileReader("nuevo.txt");
    /* Esta clase sólo nos permite leer carácter a carácter, el cual lo devuelve con su entero
     * que representa su codificación ASCII.
     * Si queremos transformarlo en carácter hay que hacer una transformación
     */
    car = fichLect.read();
    // Cuando se ha leído todo el fichero se devuelve -1
    while (car != -1) {
        cara = (char) car;
        cadenaLeida = cadenaLeida + cara;
        car = fichLect.read();
    }
    // Cerrar fichero, importante!
    fichLect.close();
    System.out.println("\nLa frase leída del fichero es: \"" + cadenaLeida + "\"");
} catch (IOException e) {
    // TODO Auto-generated catch block
    System.out.println("No se ha podido manipular el fichero");
    e.printStackTrace();
}
```

¿Se puede Mejorar?

IMPORTANTE. Si escribimos sobre un fichero que ya existe, borrará todo lo que tenía anteriormente almacenado. Si queremos añadir caracteres al final debemos de usar:

```
FileWriter ficheroTexto = new FileWriter(fichero, true)
```

AC1.5 – Crea un programa que lea el contenido de un fichero de texto, carácter a carácter, y lo muestre por pantalla. Por ejemplo el propio fichero **Main.java**.

AC1.6 – Crea un programa que cuente el número de líneas y de espacios en blanco de un fichero.

AC1.7 – Crea un programa que escriba en el fichero **"quijote.txt"** el siguiente texto. *"En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor."*

AC1.8 – Crea un programa que nos pida por teclado nombres de persona, los cuales irá añadiendo a un fichero de texto, un nombre por fila. Nos seguirá pidiendo nombres hasta que pongamos la palabra salir. Si salimos del programa y volvemos a ejecutarlo debería de seguir añadiendo nombres en el mismo fichero a partir de los que ya había introducido.

AC1.9 – Modifica el programa anterior para que antes de añadir un nuevo nombre al fichero verifique que NO esté ya en el fichero, si está lo notificamos con un mensaje y si no está pues lo añadimos. Solo podemos usar las clases `FileReader` y `FileWriter`.

5.2. BufferedReader / BufferedWriter. Lectura y escritura por líneas

Uno de los problemas que hay con las clases `FileReader` y `FileWriter` es que solamente leen/escriben carácter a carácter y esto a veces dificulta y ralentiza el desarrollo.

Las clases **`BufferedReader`** y **`BufferedWriter`** se pueden utilizar sobre las clases `FileReader` y `FileWriter` u otros flujos de caracteres para realizar operaciones de entrada/salida con un buffer, en lugar de carácter a carácter.

La clase **`BufferedReader`** dispone del método **`readLine()`**, que lee una línea del fichero y la devuelve, o devuelve **`null`** si no hay nada que leer o se llega al final del fichero. Para construir un objeto

Ejemplo: Escritura desde cero

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class EscribeFichero {
    public static void main(String[] args) {
        try (BufferedWriter writer = new BufferedWriter(
            new FileWriter("prueba.txt", StandardCharsets.UTF_8))) {
            for (int i = 1; i <= 10; i++) {
                writer.write("Línea " + i);
                writer.newLine();
            }
        } catch (IOException e) {
            System.err.println("Error de escritura: " + e.getMessage());
        }
    }
}
```

Ejemplo: Lectura línea a línea

```
try (BufferedReader reader = new BufferedReader(
    new FileReader("prueba.txt", StandardCharsets.UTF_8))) {
    String linea;
    while ((linea = reader.readLine()) != null) {
        System.out.println(linea);
    }
}
```

5.3. Sobrescribir o añadir

FileWriter sobrescribe el fichero por defecto. Para añadir al final se usa el modo **append**. Antes de elegirlo, piensa si el formato admite concatenación segura.

```
try (BufferedWriter writer = new BufferedWriter(
    new FileWriter("nombres.txt", StandardCharsets.UTF_8, true))) {
    writer.write("Ada Lovelace");
    writer.newLine();
}
```

5.4. Clase Scanner

Podríamos usar la clase scanner para leer un fichero de texto línea por línea, puede ser más cómodo pero tiene algunas desventajas como ser más lento por lo que puede ser una **peor opción si se trata de ficheros de texto de gran tamaño**.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

class LeeFichero {
    public static void main(String[] arg) {
        File archivo = new File("prueba.txt"); // usa ruta relativa si está en la misma
        carpeta

        try (Scanner scanner = new Scanner(archivo)) {
            while (scanner.hasNextLine()) {
                System.out.println(scanner.nextLine());
            }
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

5.5. try-with-resources

Los flujos implementan **AutoCloseable**. El bloque **try-with-resources** los cierra automáticamente incluso si ocurre una excepción. Es preferible a llamar a **close()** manualmente en distintos caminos de ejecución.

```
try (BufferedWriter writer = new BufferedWriter(
    new FileWriter("nombres.txt", StandardCharsets.UTF_8, true))) {
    writer.write("Ada Lovelace");
    writer.newLine();
} catch (IOException e) {
    System.err.println("No se pudo escribir el fichero: " + e.getMessage());
}
```

AC1.10 – Crea un programa que lea su código, línea a línea, y muestre su contenido por pantalla. Por ejemplo el propio fichero **Main.java**.

AC1.11 – Crea un programa que guarde en un fichero con nombre **primos.txt** los números primos que hay entre 1 y 500.

AC1.12 – Crea un programa que lea el fichero creado en el ejercicio anterior y que muestre los números por pantalla.

AC1.13 – Crea un programa que añada al final del fichero anterior, el resultado obtenido de realizar la suma acumulada de los números en las líneas pares, y restarles los de las líneas impares. Por ejemplo, si tenemos el fichero: 2 3 5 7. El resultado será: $2 - 3 + 5 - 7 = -3$

AC1.14 – Crea un programa que guarde en un fichero el contenido de otros dos ficheros, de tal forma que en el fichero resultante aparezcan las líneas de los primeros dos ficheros entre mezcladas, es decir, la primera línea será del primer fichero, la segunda será del segundo fichero, la tercera será la siguiente del primer fichero, etc. Hay que tener en cuenta que los ficheros de donde se van cogiendo las líneas pueden tener tamaños diferentes.

AC1.15 – Crea un programa que sea capaz de ordenar alfabéticamente las palabras contenidas en un fichero de texto. El nombre del fichero que contiene las palabras se debe pasar como argumento en la línea de comandos. El nombre del fichero resultado debe ser el mismo que el original añadiendo la coletilla **sort**, por ejemplo **palabras_sort.txt**. Suponemos que cada palabra ocupa una línea.

AC1.16 – Crea un programa que tenga un menú con el siguiente aspecto:

- 1.- **Añadir un usuario.**
- 2.- **Mostrar todos los usuarios añadidos.**
- 3.- **Salir.**

Si se pulsa la **opción 1**, deberá de preguntar el nombre, la edad y el videojuego favorito. Esa información la meterás en un String separando los datos por ":" (nombre:edad:videojuego) y lo añadirás en una nueva línea del fichero. La **primera línea** del fichero debe ser -> **nombre:edad:videojuego**.

Si se pulsa la **opción 2**, mostrará los datos de todos los usuarios pero de uno en uno debiendo pulsar "s", para pasar al siguiente o "n" para no seguir mostrando más usuarios. Los usuarios los mostrarás con el siguiente formato:

Nombre: Luis López

Edad: 23

Videojuego: Call Of Duty

AC1.17 – Crea un programa que muestre el siguiente menú:

- 1.- Cargar datos del fichero
- 2.- Guardar los datos en el fichero
- 3.- Añadir un nuevo Usuario
- 4.- Mostrar los datos de los usuarios
- 5.- Salir

1, carga los datos personales del fichero y los mete en un ArrayList de objetos de tipo Usuario(Nombre, NIF y Edad)

2, guarda el ArrayList de objetos de Usuarios en el fichero(sobreescribiendo lo que está almacenado).

3, añade un nuevo usuario en el ArrayList

4, muestra el contenido del ArrayList.

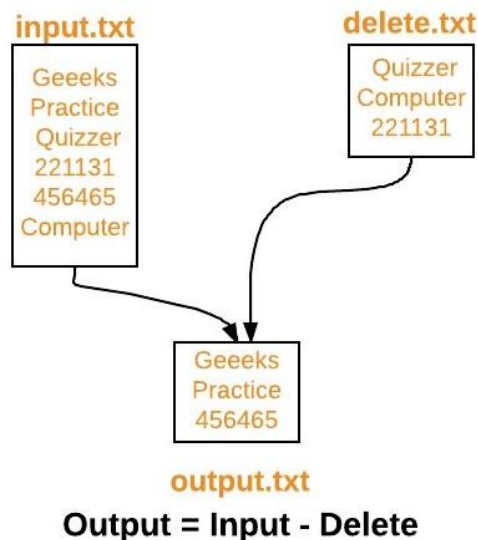
AC1.18 – Crea un programa que le pasemos un fichero “.txt” por parámetro y una opción:

“-e” encripta el fichero, creando un nuevo fichero encriptado .enc

“-d” describe el fichero .enc, que le hemos pasado por parámetro.

“-h” o cualquier otra cosa muestra un menú de ayuda de como usarlo y sale.

AC1.19 – Crea un programa al que le pasemos un fichero de texto ‘input.txt’ y otro ‘borrar.txt’ tiene que guardar la diferencia del contenido en otro llamado ‘output.txt’. Es decir, almacenar las frases que no existen en ‘borrar.txt’ pero si en ‘input.txt’



6. Ficheros binarios y tipos primitivos

Los ficheros binarios almacenan secuencias de dígitos binarios que no son legibles directamente por el usuario. Tienen la **ventaja** de que ocupan menos espacio en disco. En Java las dos clases que permiten trabajar con ficheros binarios son **FileInputStream** y **FileOutputStream**, las cuales crean un enlace entre el flujo de bytes y el fichero, permitiéndonos leer y escribir bytes. **DataInputStream** y **DataOutputStream** añaden métodos para tipos primitivos y cadenas UTF.

6.1. FileInputStream / FileOutputStream

Class FileInputStream

```
java.lang.Object
  java.io.InputStream
    java.io.FileInputStream
```

Class FileOutputStream

```
java.lang.Object
  java.io.OutputStream
    java.io.FileOutputStream
```

Método	Explicación
FileInputStream(File fichero).	Lanza la excepción FileNotFoundException si el fichero pasado no existe
FileInputStream(String fichero)	Lanza la excepción FileNotFoundException si el fichero pasado no existe
int read()	Devuelve el código ASCII del siguiente byte que hay después de donde está situado el puntero del fichero . Dicho puntero se va moviendo secuencialmente por el fichero, según vamos leyendo los bytes. Devuelve -1 si no hay ningún byte más que leer
int read(char[] buf)	Lee hasta buf.length caracteres de datos de la matriz buf pasada como parámetro
int read(char[] buf, int desplazamiento, int n)	Lee hasta n caracteres de datos de una matriz buf comenzando por buf[desplazamiento] y devuelve el número leído de caracteres.

Método	Explicación
FileOutputStream(String nombreFich) FileOutputStream(String nombreFich, boolean append)	Recibe como parámetro el nombre del fichero a abrir y borra el contenido previo del fichero comenzando a escribir desde el principio. Si append es true, se sitúa al final del fichero para añadir contenido desde el final.
FileOutputStream(File fichero) FileOutputStream(File fichero, boolean append)	Recibe como parámetro un objeto File que representa al fichero con el que queremos trabajar y borra el contenido previo del fichero comenzando a escribir desde el principio. Si append es true, se sitúa al final del fichero para añadir contenido desde el final.
void write(int byte)	Escribe el byte que recibe como argumento en el fichero
void write(char[] buf)	Escribe un array de caracteres.
void write(char[] buf, int desplazamiento, int n)	Escribe n caracteres de datos de una matriz buf comenzando por buf[desplazamiento].

Ejemplo: Copiar un fichero binario

```
try (InputStream in = new FileInputStream("imagen.jpg");
    OutputStream out = new FileOutputStream("imagen_copia.jpg")) {

    byte[] buffer = new byte[8192];
    int leidos;
    while ((leidos = in.read(buffer)) != -1) {
        out.write(buffer, 0, leidos);
    }
}
```

MEJORA moderna. Para una copia simple, **Files.copy(origen, destino, REPLACE_EXISTING)** es más breve. El ejemplo con streams es útil para comprender el flujo de bytes y para procesos donde queremos transformar datos durante la copia.

AC1.20 – Crea un programa que lea un fichero de imagen, “imagen.jpg”, y lo copie en otro fichero con el nombre “imagen_copia.jpg”. Verifica que el tamaño de la original y la copia coinciden. Puedes usar `Files.size()`.

6.2. DataInputStream y DataOutputStream

Para leer y escribir datos de tipos primitivos (como boolean, int, long, float, double, char, etc.), se utilizan las clases **DataInputStream** y **DataOutputStream**. Además de los métodos **read()** y **write()**, estas clases proporcionan métodos para la lectura y escritura de tipos primitivos de un modo independiente de la máquina. Algunos de los métodos disponibles son:

MÉTODOS PARA LECTURA	MÉTODOS PARA ESCRITURA
boolean readBoolean(); byte readByte(); int readUnsignedByte(); int readUnsignedShort(); short readShort(); char readChar(); int readInt(); long readLong(); float readFloat(); double readDouble(); String readUTF();	void writeBoolean(boolean v); void writeByte(int v); void writeBytes(String s); void writeShort(int v); void writeChars(String s); void writeChar(int v); void writeInt(int v); void writeLong(long v); void writeFloat(float v); void writeDouble(double v); void writeUTF(String str);

(UTF, Unicode Transmission Format, se utiliza para transmitir los caracteres de un String de 16 bits codificada en 8 bits.)

Escritura	Lectura	Bytes típicos
writeBoolean	readBoolean	1 byte en DataOutputStream
writeByte	readByte	1
writeShort	readShort	2
writeChar	readChar	2
writeInt	readInt	4
writeLong	readLong	8
writeFloat	readFloat	4
writeDouble	readDouble	8
writeUTF	readUTF	Variable; incluye longitud

writeUTF(String s): Este método escribe una cadena de texto en un formato binario especial llamado Modified UTF-8. Lo primero que guarda son dos bytes con la **longitud en bytes** de la cadena, y después los propios caracteres codificados. La **ventaja** es que la longitud queda registrada automáticamente y no tienes que preocuparte de contar los caracteres ni de añadir un terminador. Es cómodo y seguro si vas a leer la información con readUTF..

readUTF(): Este método lee del flujo de entrada una cadena que haya sido escrita previamente con writeUTF. Lo primero que hace es leer esos dos bytes de longitud para saber cuántos caracteres vienen después y luego decodifica los bytes al formato de String de Java. De esta manera recuperas exactamente la cadena que se guardó, sin ambigüedades.

ATENCIÓN Usando DataOutputStream.writeBoolean() puede pensarse que almacena un bit, pero escribe un byte. Para calcular registros reales hay que usar el comportamiento del formato concreto.

Ejemplo. Escritura de registros con tipos primitivos

```
// Instanciamos nuestro flujo de escritura de datos (bytes)
DataOutputStream out = new DataOutputStream(new
    FileOutputStream("nuevo_dir/facturas"));

// Preparamos las variables y los datos que almacenamos en nuestro fichero.
double[] precios = { 19.99, 9.99, 15.99, 3.99, 4.99 };
int[] unidades = { 12, 8, 13, 29, 50 };
String[] objetos = { "Disco ELCD",
    "Gorra Nike",
    "Casmietta Tecnyconta",
    "Pin ALF",
    "Libro Java" };

// Escribimos según este formato: precio    unidades    objeto
for (int i = 0; i < precios.length; i++) {
    out.writeDouble(precios[i]);
    out.writeChar('\t');
    out.writeInt(unidades[i]);
    out.writeChar('\t');
    //El método UTF es el que escribe y lee de tipo String.
    out.writeUTF(objetos[i]);
    out.writeChar('\n');
}
// CERRAMOS EL FLUJO!
out.close();
```

Ejemplo. Lectura de registros. (Debe respetar exactamente el mismo orden y tipos)

```
// Abrimos el fichero, esta vez con un flujo modo lectura
DataInputStream in = new DataInputStream(new
    FileInputStream("nuevo_dir/facturas"));
```

```
double precio;
int unidad;
String desc;
double total = 0.0;
```

```
try {
    while (true) {

        precio = in.readDouble();
        in.readChar(); // leemos el tab
        unidad = in.readInt();
        in.readChar(); // leemos el tab
        desc = in.readUTF();
        in.readChar();
        System.out.println("Has pedido " +
            unidad + " unidades de " +
            desc + " cada una vale " + precio + " EUR");
        total = total + unidad * precio;
    }
} catch (EOFException e) {
```

```
/*
 * En este caso, en vez de controlar si nos devuelve -1, esperamos a que lance la excepción
 * y la recuperamos.
 */
```

```
System.out.println("Te has dejado en total: " + total);
```

```
}
in.close();
}
```

```
Has pedido 12 unidades de Disco ELCD cada una vale 19.99 EUR
Has pedido 8 unidades de Gorra Nike cada una vale 9.99 EUR
Has pedido 13 unidades de Casmietta Tecnyconta cada una vale 15.99 EUR
Has pedido 29 unidades de Pin ALF cada una vale 3.99 EUR
Has pedido 50 unidades de Libro Java cada una vale 4.99 EUR
Te has dejado en total: 892.8800000000001
```

Ejemplo. Escritura de registros con tipos primitivos

```
Factura[] facturas = {
    new Factura("Disco ELCD", 12, 19.99),
    new Factura("Gorra Nike", 8, 9.99),
    new Factura("Camiseta Tecnyconta", 13, 15.99),
    new Factura("Pin ALF", 29, 3.99),
    new Factura("Libro Java", 50, 4.99)
};

try (DataOutputStream out = new DataOutputStream(
    new BufferedOutputStream(new FileOutputStream("facturas.dat")))) {
    for (Factura f : facturas) {
        out.writeUTF(f.objeto());
        out.writeInt(f.unidades());
        out.writeDouble(f.precio());
    }
}
```

Ejemplo. Lectura de registros. (Debe respetar exactamente el mismo orden y tipos)

```
try (DataInputStream in = new DataInputStream(
    new BufferedInputStream(new FileInputStream("facturas.dat")))) {

    while (true) {.    // Se rompe cuando salga un EOFException
        String objeto = in.readUTF();
        int unidades = in.readInt();
        double precio = in.readDouble();
        System.out.printf("Objeto:%-20s | Unidades:%2d | Precio:%.2f €\n", objeto, unidades, precio);
    }
} catch (EOFException fin) {
    System.out.println("Fin de fichero.");
} catch (IOException e) {
    System.err.println("Error de lectura: " + e.getMessage());
}
```

```
Objeto: Disco ELCD      | Unidades: 12 | Precio: 19,99 ?
Objeto: Gorra Nike     | Unidades:  8 | Precio:  9,99 ?
Objeto: Camiseta Tecnyconta | Unidades: 13 | Precio: 15,99 ?
Objeto: Pin ALF        | Unidades: 29 | Precio:  3,99 ?
Objeto: Libro Java     | Unidades: 50 | Precio:  4,99 ?
Fin de fichero alcanzado.
```

REGLA de ORO Si se escribe String → int → double, se debe leer String → int → double. Un fichero binario propio necesita una especificación de formato; sin ella, los bytes no se pueden interpretar de forma fiable.

AC1.21 – Crea un programa que lea desde un fichero de texto llamado **facturas.txt** los datos de la factura de compra y los almacene en un fichero binario llamado **facturas.dat**. La estructura del fichero factura.txt es: artículo:unidades:precioUnitario.

Luego debes recuperar los datos del fichero facturas.dat y mostrarlo por pantalla.

AC1.22 – Crea un programa que guarde en un fichero con nombre numerosArmstrong.dat los números de entre 1 y 1000 que sean números de Armstrong. Un número de Armstrong es aquel que es igual a la suma de sus propios dígitos elevados a la potencia de la cantidad de dígitos.

Por ejemplo, 370 es un número de Armstrong ya que $370 = 3^3 + 7^3 + 0^3$.

AC1.23 – Crea un programa que lea del fichero numerosArmstrong.dat generado en el ejercicio anterior, y muestre por pantalla aquellos que sean pares.

Para hacer estos dos ejercicios, utilizad las clases **DataInputStream** y **DataOutputStream**

7. Serialización de objetos. `ObjectInputStream` / `ObjectOutputStream`.

En Java, para poder guardar un objeto en un fichero binario, dicho objeto tiene que **implementar la interfaz `Serializable`**, que permite que una clase pueda ser transformada a una secuencia de bytes.

La serialización de objetos de Java permite tomar cualquier objeto que implemente la interfaz `Serializable` y **convertirlo en una secuencia de bits**, que puede ser posteriormente restaurada para regenerar el objeto original. Para leer y escribir objetos serializables a un stream se utilizan las **clases `ObjectInputStream` y `ObjectOutputStream`**

Los **métodos** más usados serán **`readObject()` y `writeObject()`**.

Es cómoda para ejercicios y herramientas Java, pero no es un formato de intercambio recomendable a largo plazo: queda acoplado a las clases Java y requiere cuidado de seguridad y compatibilidad.



Figura 2. Flujo conceptual de serialización y deserialización.

```

public class Persona implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String nombre;
    private final int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años)";
    }
}
  
```

`serialVersionUID`. Es un identificador de versión de la clase para la serialización. Cuando guardas una `Persona`, Java incluye este identificador. Al recuperarla, compara el identificador del fichero con el de la clase actual: **Si coinciden**, continúa la deserialización, siempre que los cambios sean compatibles. **Si son distintos**, lanza una **`InvalidClassException`**.

Si no lo declaras, Java calcula uno automáticamente **según la estructura de la clase**. Un cambio en ella puede alterar ese valor y provocar que no puedas leer objetos guardados anteriormente.

```

public class FicheroPersonas {
    private final String ruta;

    public FicheroPersonas(String ruta) {
        this.ruta = ruta;
    }

    public void guardar(ArrayList<Persona> personas) throws IOException {
        // Se reemplaza el contenido del fichero por la lista completa.
        try (ObjectOutputStream salida = new ObjectOutputStream(new FileOutputStream(ruta))) {
            salida.writeObject(personas);
        }
    }

    @SuppressWarnings("unchecked")
    public ArrayList<Persona> cargar() throws IOException, ClassNotFoundException {
        try (ObjectInputStream entrada = new ObjectInputStream(new FileInputStream(ruta))) {
            // El fichero debe contener el ArrayList<Persona> escrito por guardar().
            return (ArrayList<Persona>) entrada.readObject();
        }
    }
}

```

CABECERAS al hacer **APPEND**, `ObjectOutputStream` escribe una cabecera al construirse. Abrir el mismo fichero en **append** y crear otro `ObjectOutputStream` puede insertar una segunda cabecera y provocar `StreamCorruptedException` al leer. Para clase, es más simple serializar una colección completa y reescribirla; si se desea **append** de objetos individuales, hay que controlar la cabecera con una subclase especializada.

SEGURIDAD No deserialices con `ObjectInputStream` datos binarios de origen no confiable. La **deserialización** Java ha sido históricamente una **superficie de ataque**. En aplicaciones reales se prefieren formatos explícitos como **JSON/XML** con validación.

AC1.24 – Personas Serializables. Crea una clase `Persona` **Serializable**, guarda una lista de personas y recupérala. Después modifica el fichero añadiendo una nueva persona mediante **carga-modificación-guardado**.

AC1.25 – Crea un programa que almacene los datos de clientes. Para cliente se quiere guardar: código del cliente (`Int`), nombre(`String`), dirección(`String`), saldo(`Double`), cuenta(`String`. 'Al día', 'Atrasada', 'Deudor'). El programa deberá de poder guardar y recuperar a los clientes de la empresa. Además debe poder consultar a los morosos a través de la función: `ArrayList<Cliente> devolverMorosos (File fichero);`

8. Ficheros de acceso aleatorio: RandomAccessFile

En Java la clase **RandomAccessFile** permite leer y escribir en posiciones arbitrarias de un fichero mediante un puntero. No pertenece a las jerarquías Reader/Writer ni InputStream/OutputStream. Implementa las interfaces DataInput y DataOutput, por eso ofrece métodos readInt(), writeDouble(), readUTF(), etc.

Método	Explicación
RandomAccessFile(String nombre, String modo)	nombre es el nombre del fichero y modo es el argumento que determina si el contenido del fichero se va a poder solo leer ("r") o leer y escribir ("rw").
RandomAccessFile(File fichero, String modo)	fichero es el objeto fichero y modo es el argumento que determina si el contenido del fichero se va a poder solo leer ("r") o leer y escribir ("rw").

La clase **RandomAccessFile** maneja un **puntero o cursor** que indica la posición actual en el fichero. Cuando se **abre un fichero**, el puntero se **coloca en 0**, es decir, apuntando al principio del mismo. Una vez abierto un fichero, se pueden utilizar sus métodos read() y write() para cada tipo de dato y las sucesivas llamadas a estos métodos **read()** y **write()** ajustan el puntero según la cantidad de bytes leídos o escritos.

Método	Explicación
long getFilePointer()	Devuelve la posición actual del puntero del fichero
void seek(long posicion)	Coloca el puntero del fichero en una posición determinada desde el comienzo del mismo
long length()	Devuelve el tamaño del fichero en bytes y marca el final del fichero
int skipBytes(int desplazamiento)	Desplaza el puntero del fichero desde la posición actual el número de bytes indicados en desplazamiento.

Métodos para lectura	Métodos para escritura
boolean readBoolean()	void writeBoolean(boolean b)
byte readByte()	void writeByte(int i)
int readUnsignedByte()	void writeBytes(String s)
int readUnsignedShort()	void writeShort(int i)
short readShort()	void writeChars(String s)
char readChar()	void writeChar(int i)
int readInt()	void writeInt(int i)
long readLong()	void writeLong(long l)
float readFloat()	void writeFloat(float f)
double readDouble()	void writeDouble(double d)
String readUTF()	void writeUTF(String s)

Antes de empezar a trabajar con datos de acuerdo con su tamaño, hay que tener cuenta diferencias de codificación entre `writeChars()` y `writeUTF()`

- **WriteChars()** escribe el String como secuencia de caracteres de **2 bytes** cada uno (**UTF-16**), y no se almacena información de ningún tipo sobre la codificación. Por ejemplo, "Gatito" se almacenará como 12 bytes (6 caracteres por 2 bytes)

- **WriteUTF()** escribe el String en codificación **UTF-8**, es decir, primero escribe **una cabecera de 2 bytes indicando el número de bytes a escribir**, y después almacena los bytes en codificación **UTF8**. Esto resulta en un almacenamiento más eficiente. Por ejemplo, "Gatito" se almacenará como 8 bytes (2 de la cabecera y 6 por cada carácter).

8.1. Diseñar un registro de tamaño fijo

A continuación se hará un ejemplo tanto de escritura como lectura en un fichero de acceso aleatorio, teniendo en cuenta primero la siguiente consideración.

Cada tipo de dato tiene un tamaño concreto:		
boolean (1 bit)	byte (1 byte)	carácter Unicode (2 bytes)
short (2 bytes)	int (4 bytes)	long (8 bytes)
float (4 bytes)	double (8 bytes)	

Para **calcular la posición de un registro necesitamos que todos ocupen lo mismo**. En el ejemplo clásico: `id int (4) + apellido de 10 char escritos con writeChars (10×2=20) + departamento int (4) + salario double (8) = 36 bytes`.

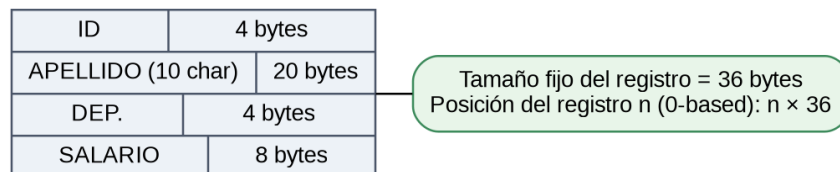


Figura 4. Ejemplo de registro fijo para acceso directo.

```
private static final int CHARS_APELLIDO = 10;
private static final int BYTES_REGISTRO = 4 + (CHARS_APELLIDO * 2) + 4 + 8;

private static long posicionRegistro(int indice) {
    return (long) indice * BYTES_REGISTRO;
}
```

8.2. Escribir y leer registros

Índice frente a id Si los ids empiezan en 1, la posición del empleado con id = 1 es 0. Por tanto: $\text{posicion} = (\text{id} - 1) * \text{BYTES_REGISTRO}$.

```
static Empleado buscarPorId(Path fichero, int id) throws IOException {
    if (id <= 0) return null;

    try (RandomAccessFile raf = new RandomAccessFile(fichero.toFile(), "r")) {
        long pos = (id - 1L) * BYTES_REGISTRO;
        if (pos + BYTES_REGISTRO > raf.length())
            return null;

        raf.seek(pos);
        int idLeido = raf.readInt();
        String apellido = leerApellidoFijo(raf);
        int departamento = raf.readInt();
        double salario = raf.readDouble();

        return idLeido <= 0 ? null
            : new Empleado(idLeido, apellido, departamento, salario);
    }
}
```

```
static void escribirApellidoFijo(RandomAccessFile raf, String apellido) throws IOException {
    String limpio = apellido == null ? "" : apellido;
    if (limpio.length() > CHARS_APELLIDO) {
        limpio = limpio.substring(0, CHARS_APELLIDO);
    }
    // String ajustado = String.format("%-10s", "Pérez");
    String ajustado = String.format("%-" + CHARS_APELLIDO + "s", limpio);
    raf.writeChars(ajustado);
}

static String leerApellidoFijo(RandomAccessFile raf) throws IOException {
    char[] chars = new char[CHARS_APELLIDO];
    for (int i = 0; i < chars.length; i++) {
        chars[i] = raf.readChar();
    }
    return new String(chars).trim();
}
```

AC1.26 – Crea un programa que escriba, liste y modifique los datos de empleados con el siguiente formato id + apellido(10 char) + departamento + salario. Usa constantes en vez de usar números mágicos. Usa las clases Empleado y EmpleadoAccesoAleatorioRepositorio.

AC1.27 – Crea un programa que amplíe la funcionalidad del ejercicio anterior para hacer un CRUD sobre empleados. Implementalo aplicando el patrón MVC.

9. MVC. Modelo – Vista – Controlador

